BABU BANARASI DAS UNIVERSITY



2026-2027

SCHOOL OF COMPUTER APPLICATIONS

No SQL Project On

Student Management System Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 25th September

Submitted by:

Alok Pratap Singh

Roll no: 57

University Roll no: 1250258065

ACKNOWLEDGEMENT

I would like to sincerely thank my teacher, Mr. Harendra Singh, for giving me the chance to work on this NoSQL project titled “Student Management System Using MongoDB”.

Working on this project helped me understand the core ideas behind NoSQL databases and showed me how MongoDB is used to create, read, update, sort and delete data. I also learned how query operators such as greater than, less than, equal to, AND, and OR are applied while searching for documents.

I am grateful to my teacher for the guidance and support given throughout the completion of this project.

INTRODUCTION

MongoDB is a widely used NoSQL database management system that keeps data in the form of documents. Unlike traditional relational databases, it does not force data into fixed tables made of rows and columns.

In this project, a Student Management System has been built using MongoDB. The database stores details of 50 students, including their roll number, name, age, marks, and city. Several MongoDB operations have been carried out on this data, such as inserting, displaying, updating, deleting, sorting, limiting, counting, and searching records. The comparison operators $eq, $gt, $lt, $gte, and $lte, along with the logical operators $and and $or, have also been demonstrated.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and a collection.
- To insert 50 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting, limiting and filtering.
- To understand AND and OR operations.

1. CREATE DATABASE

MongoDB Query:

```
use StudentDB
```

Explanation: Creates the StudentDB database if it does not exist, or switches to it.

```
>_ MONGOSH                                                    test>

> use StudentDB
< switched to db StudentDB
```

2. CREATE COLLECTION

MongoDB Query:

```
db.createCollection("students")
```

Explanation: Creates a collection named students.

```
>_ MONGOSH                                                StudentDB>

> db.createCollection("students")
< { ok: 1 }
```

3. INSERT 50 STUDENTS

MongoDB Query:

```
db.students.insertMany([
  {roll:1, name:"Aryan", age:20, marks:78, city:"Varanasi"},
  {roll:2, name:"Sakshi", age:19, marks:92, city:"Prayagraj"},
  {roll:3, name:"Nikhil", age:21, marks:66, city:"Gorakhpur"},
  {roll:4, name:"Priya", age:19, marks:85, city:"Bareilly"},
  {roll:5, name:"Rohan", age:20, marks:73, city:"Dehradun"},
  {roll:6, name:"Anushka", age:18, marks:90, city:"Varanasi"},
  {roll:7, name:"Vivek", age:21, marks:58, city:"Patna"},
  {roll:8, name:"Muskaan", age:19, marks:81, city:"Jaipur"},
  {roll:9, name:"Pranav", age:20, marks:69, city:"Prayagraj"},
  {roll:10, name:"Shreya", age:18, marks:94, city:"Varanasi"},
  {roll:11, name:"Aman", age:21, marks:87, city:"Bhopal"},
  {roll:12, name:"Naina", age:19, marks:76, city:"Gorakhpur"},
  {roll:13, name:"Tushar", age:20, marks:62, city:"Bareilly"},
  {roll:14, name:"Ritika", age:18, marks:90, city:"Prayagraj"},
  {roll:15, name:"Ayush", age:19, marks:55, city:"Dehradun"},
  {roll:16, name:"Komal", age:20, marks:83, city:"Varanasi"},
  {roll:17, name:"Dhruv", age:21, marks:71, city:"Patna"},
  {roll:18, name:"Aarohi", age:18, marks:96, city:"Jaipur"},
  {roll:19, name:"Sumit", age:20, marks:64, city:"Bhopal"},
  {roll:20, name:"Ananya", age:19, marks:88, city:"Prayagraj"},
  {roll:21, name:"Lakshya", age:21, marks:79, city:"Varanasi"},
  {roll:22, name:"Bhumika", age:18, marks:60, city:"Gorakhpur"},
  {roll:23, name:"Pankaj", age:20, marks:84, city:"Bareilly"},
  {roll:24, name:"Vaishnavi", age:19, marks:91, city:"Dehradun"},
  {roll:25, name:"Hemant", age:21, marks:67, city:"Patna"},
  {roll:26, name:"Tanmay", age:20, marks:89, city:"Jaipur"},
  {roll:27, name:"Charu", age:18, marks:57, city:"Varanasi"},
  {roll:28, name:"Gautam", age:19, marks:74, city:"Bhopal"},
  {roll:29, name:"Radhika", age:20, marks:95, city:"Prayagraj"},
```

```
  {roll:30, name:"Sagar", age:21, marks:82, city:"Gorakhpur"},
  {roll:31, name:"Neha", age:18, marks:77, city:"Bareilly"},
  {roll:32, name:"Yuvraj", age:19, marks:90, city:"Varanasi"},
  {roll:33, name:"Jyoti", age:20, marks:53, city:"Patna"},
  {roll:34, name:"Kunal", age:21, marks:86, city:"Dehradun"},
  {roll:35, name:"Aisha", age:19, marks:98, city:"Jaipur"},
  {roll:36, name:"Rahul", age:20, marks:68, city:"Bhopal"},
  {roll:37, name:"Shruti", age:18, marks:80, city:"Prayagraj"},
  {roll:38, name:"Mayank", age:19, marks:72, city:"Gorakhpur"},
  {roll:39, name:"Divya", age:21, marks:93, city:"Varanasi"},
  {roll:40, name:"Sparsh", age:18, marks:65, city:"Bareilly"},
  {roll:41, name:"Pallavi", age:20, marks:85, city:"Patna"},
  {roll:42, name:"Ujjwal", age:19, marks:75, city:"Dehradun"},
  {roll:43, name:"Nisha", age:18, marks:61, city:"Jaipur"},
  {roll:44, name:"Chirag", age:21, marks:97, city:"Varanasi"},
  {roll:45, name:"Garima", age:19, marks:70, city:"Bhopal"},
  {roll:46, name:"Himanshu", age:20, marks:87, city:"Prayagraj"},
  {roll:47, name:"Rashi", age:18, marks:63, city:"Gorakhpur"},
  {roll:48, name:"Zoya", age:19, marks:89, city:"Bareilly"},
  {roll:49, name:"Atul", age:21, marks:59, city:"Dehradun"},
  {roll:50, name:"Bhavya", age:20, marks:84, city:"Patna"}
])
```

Explanation: Inserts 50 student documents into the students collection in one go.

```
>_ MONGOSH                                                    StudentDB>

> db.students.insertMany([
  {roll:1, name:"Aryan", age:20, marks:78, city:"Varanasi"},
  {roll:2, name:"Sakshi", age:19, marks:92, city:"Prayagraj"},
  {roll:3, name:"Nikhil", age:21, marks:66, city:"Gorakhpur"},
  {roll:4, name:"Priya", age:19, marks:85, city:"Bareilly"},
  {roll:5, name:"Rohan", age:20, marks:73, city:"Dehradun"},
  {roll:6, name:"Anushka", age:18, marks:90, city:"Varanasi"},
  {roll:7, name:"Vivek", age:21, marks:58, city:"Patna"},
  {roll:8, name:"Muskaan", age:19, marks:81, city:"Jaipur"},
  {roll:9, name:"Pranav", age:20, marks:69, city:"Prayagraj"},
  {roll:10, name:"Shreya", age:18, marks:94, city:"Varanasi"},
  {roll:11, name:"Aman", age:21, marks:87, city:"Bhopal"},
  {roll:12, name:"Naina", age:19, marks:76, city:"Gorakhpur"},
  ...
])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6b3e8a17c94f20b6d1a31c05'),
      '1': ObjectId('6b3e8a17c94f20b6d1a31c06'),
      ...
      '49': ObjectId('6b3e8a17c94f20b6d1a31c36')
    }
  }
```

4. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.students.find()
```

Explanation: Retrieves documents from the students collection.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find()
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c05'),
  roll: 1,
  name: 'Aryan',
  age: 20,
  marks: 78,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c06'),
  roll: 2,
  name: 'Sakshi',
  age: 19,
  marks: 92,
  city: 'Prayagraj'
}
... (48 more documents; output cropped)
```

5. EQUAL TO — $eq

MongoDB Query:

```
db.students.find({marks: {$eq: 90}})
```

Explanation: Finds students whose marks are exactly 90.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({marks: {$eq: 90}})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0a'),
  roll: 6,
  name: 'Anushka',
  age: 18,
  marks: 90,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c12'),
  roll: 14,
  name: 'Ritika',
  age: 18,
  marks: 90,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c24'),
  roll: 32,
  name: 'Yuvraj',
  age: 19,
  marks: 90,
  city: 'Varanasi'
}
```

6. GREATER THAN — $gt

MongoDB Query:

```
db.students.find({marks: {$gt: 88}})
```

Explanation: Finds students whose marks are greater than 88.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({marks: {$gt: 88}})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c06'),
  roll: 2,
  name: 'Sakshi',
  age: 19,
  marks: 92,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0a'),
  roll: 6,
  name: 'Anushka',
  age: 18,
  marks: 90,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0e'),
  roll: 10,
  name: 'Shreya',
  age: 18,
  marks: 94,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c12'),
  roll: 14,
  name: 'Ritika',
  age: 18,
  marks: 90,
  city: 'Prayagraj'
}
... (9 more documents; output cropped)
```

7. LESS THAN — $lt

MongoDB Query:

```
db.students.find({marks: {$lt: 60}})
```

Explanation: Finds students whose marks are less than 60.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({marks: {$lt: 60}})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0b'),
  roll: 7,
  name: 'Vivek',
  age: 21,
  marks: 58,
  city: 'Patna'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c13'),
  roll: 15,
  name: 'Ayush',
  age: 19,
  marks: 55,
  city: 'Dehradun'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c1f'),
  roll: 27,
  name: 'Charu',
  age: 18,
  marks: 57,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c25'),
  roll: 33,
  name: 'Jyoti',
  age: 20,
  marks: 53,
  city: 'Patna'
}
... (1 more documents; output cropped)
```

8. GREATER THAN OR EQUAL TO — $gte

MongoDB Query:

```
db.students.find({marks: {$gte: 92}})
```

Explanation: Finds students whose marks are 92 or above.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({marks: {$gte: 92}})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c06'),
  roll: 2,
  name: 'Sakshi',
  age: 19,
  marks: 92,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0e'),
  roll: 10,
  name: 'Shreya',
  age: 18,
  marks: 94,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c16'),
  roll: 18,
  name: 'Aarohi',
  age: 18,
  marks: 96,
  city: 'Jaipur'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c21'),
  roll: 29,
  name: 'Radhika',
  age: 20,
  marks: 95,
  city: 'Prayagraj'
}
... (3 more documents; output cropped)
```

9. LESS THAN OR EQUAL TO — $lte

MongoDB Query:

```
db.students.find({marks: {$lte: 65}})
```

Explanation: Finds students whose marks are 65 or below.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({marks: {$lte: 65}})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0b'),
  roll: 7,
  name: 'Vivek',
  age: 21,
  marks: 58,
  city: 'Patna'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c11'),
  roll: 13,
  name: 'Tushar',
  age: 20,
  marks: 62,
  city: 'Bareilly'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c13'),
  roll: 15,
  name: 'Ayush',
  age: 19,
  marks: 55,
  city: 'Dehradun'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c17'),
  roll: 19,
  name: 'Sumit',
  age: 20,
  marks: 64,
  city: 'Bhopal'
}
... (7 more documents; output cropped)
```

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne({roll:8}, {$set:{marks:98}})
```

Explanation: Updates the marks to 98 of the student with roll number 8.

```
>_ MONGOSH                                                    StudentDB>

> db.students.updateOne(
  {roll:8},
  {$set:{marks:98}}
)
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.students.find({roll:8})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0c'),
  roll: 8,
  name: 'Muskaan',
  age: 19,
  marks: 98,
  city: 'Jaipur'
}
```

11. DELETE ONE OPERATION

MongoDB Query:

```
db.students.deleteOne({roll:12})
```

Explanation: Deletes one document matching roll number 12.

```
>_ MONGOSH                                                    StudentDB>

> db.students.deleteOne({roll:12})
< {
    acknowledged: true,
    deletedCount: 1
  }
> db.students.find({roll:12})
<
```

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({marks: {$lt:60}})
```

Explanation: Deletes all students having marks below 60.

```
>_ MONGOSH                                                    StudentDB>

> db.students.deleteMany({marks:{$lt:60}})
< {
    acknowledged: true,
    deletedCount: 5
  }
> db.students.find({marks:{$lt:60}})
<
```

13. AND OPERATION — $and

MongoDB Query:

```
db.students.find({$and:[{marks:{$gt:75}}, {age:20}]})
```

Explanation: Finds students satisfying both conditions (marks above 75 and age 20).

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({
  $and:[
    {marks:{$gt:75}},{age:20}
  ]
})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c05'),
  roll: 1,
  name: 'Aryan',
  age: 20,
  marks: 78,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c14'),
  roll: 16,
  name: 'Komal',
  age: 20,
  marks: 83,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c1b'),
  roll: 23,
  name: 'Pankaj',
  age: 20,
  marks: 84,
  city: 'Bareilly'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c1e'),
  roll: 26,
  name: 'Tanmay',
  age: 20,
  marks: 89,
  city: 'Jaipur'
}
... (4 more documents; output cropped)
```

14. OR OPERATION — $or

MongoDB Query:

```
db.students.find({$or:[{city:"Varanasi"}, {city:"Prayagraj"}]})
```

Explanation: Finds students from Varanasi or Prayagraj.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({
  $or:[
    {city:"Varanasi"},{city:"Prayagraj"}
  ]
})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c05'),
  roll: 1,
  name: 'Aryan',
  age: 20,
  marks: 78,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c06'),
  roll: 2,
  name: 'Sakshi',
  age: 19,
  marks: 92,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0a'),
  roll: 6,
  name: 'Anushka',
  age: 18,
  marks: 90,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0d'),
  roll: 9,
  name: 'Pranav',
  age: 20,
  marks: 69,
  city: 'Prayagraj'
}
... (11 more documents; output cropped)
```

15. SORT OPERATION

A. MongoDB Query:

```
db.students.find().sort({marks: 1})
```

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find().sort({marks:1})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c1a'),
```

```
  roll: 22,
  name: 'Bhumika',
  age: 18,
  marks: 60,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c2f'),
  roll: 43,
  name: 'Nisha',
  age: 18,
  marks: 61,
  city: 'Jaipur'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c11'),
  roll: 13,
  name: 'Tushar',
  age: 20,
  marks: 62,
  city: 'Bareilly'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c33'),
  roll: 47,
  name: 'Rashi',
  age: 18,
  marks: 63,
  city: 'Gorakhpur'
}
... (40 more documents; output cropped)
```

B. MongoDB Query:

```
db.students.find().sort({marks: -1})
```

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find().sort({marks:-1})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0c'),
  roll: 8,
  name: 'Muskaan',
  age: 19,
  marks: 98,
  city: 'Jaipur'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c27'),
  roll: 35,
  name: 'Aisha',
  age: 19,
  marks: 98,
  city: 'Jaipur'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c30'),
  roll: 44,
  name: 'Chirag',
  age: 21,
```

```
  marks: 97,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c16'),
  roll: 18,
  name: 'Aarohi',
  age: 18,
  marks: 96,
  city: 'Jaipur'
}
... (40 more documents; output cropped)
```

16. LIMIT OPERATION

MongoDB Query:

```
db.students.find().limit(5)
```

Explanation: Displays only the first five documents.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find().limit(5)
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c05'),
  roll: 1,
  name: 'Aryan',
  age: 20,
  marks: 78,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c06'),
  roll: 2,
  name: 'Sakshi',
  age: 19,
  marks: 92,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c07'),
  roll: 3,
  name: 'Nikhil',
  age: 21,
  marks: 66,
  city: 'Gorakhpur'
}
... (2 more documents; output cropped)
```

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({city:"Varanasi"})
```

Explanation: Finds all students whose city is Varanasi.

```
>_ MONGOSH                                                    StudentDB>

> db.students.find({city:"Varanasi"})
< {
  _id: ObjectId('6b3e8a17c94f20b6d1a31c05'),
  roll: 1,
  name: 'Aryan',
  age: 20,
  marks: 78,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0a'),
  roll: 6,
  name: 'Anushka',
  age: 18,
  marks: 90,
  city: 'Varanasi'
}
{
  _id: ObjectId('6b3e8a17c94f20b6d1a31c0e'),
  roll: 10,
  name: 'Shreya',
  age: 18,
  marks: 94,
  city: 'Varanasi'
}
... (5 more documents; output cropped)
```

18. COUNT DOCUMENTS

MongoDB Query:

```
db.students.countDocuments()
```

Explanation: Counts documents currently present in the collection (performed after the delete operations).

```
>_ MONGOSH                                                    StudentDB>

> db.students.countDocuments()
< 44
```

MONGODB OPERATORS USED:

Operator	Meaning	Example
$eq	Equal to	marks: {$eq:90}
$gt	Greater than	marks: {$gt:88}
$lt	Less than	marks: {$lt:60}
$gte	Greater than or equal to	marks: {$gte:92}
$lte	Less than or equal to	marks: {$lte:65}
$and	All conditions true	$and:[...]
$or	At least one condition true	$or:[...]

CONCLUSION

This project gave me practical knowledge of NoSQL databases using MongoDB. A Student Management System holding student information was created, and a variety of MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to sort data in ascending and descending order, how to limit and count results, and how the comparison operators $eq, $gt, $lt, $gte, and $lte work, along with the logical operators $and and $or.

Overall, this project helped me understand how MongoDB can be used to store, manage and query student data efficiently.